

Computational Physics

Lecture 3: Numerical Integration

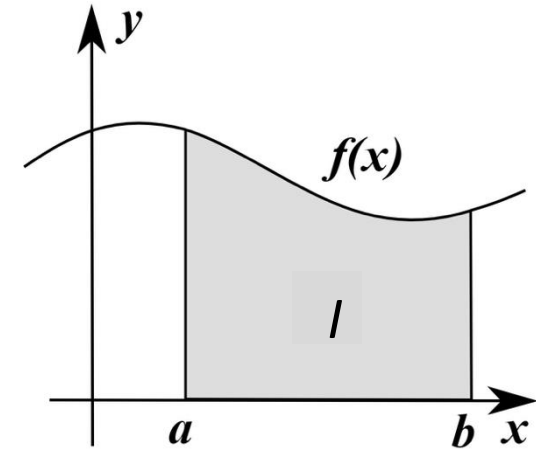
- Basic methods for numerical integration (trapezoid, Simpson)
- Gaussian quadrature
- Integrals over infinite ranges

References: *Computational Physics* by Mark Newman, Chapter 5
<https://websites.umich.edu/~mejn/cp2/chapters/chapter5.pdf>
Numerical Recipes Third Edition by W.H. Press et al.

Numerical integration

Generic problem: evaluate the following integral of a function $f(x)$ in a specified domain from $x=a$ to $x=b$

$$I(a, b) = \int_a^b f(x) \, dx. \quad \text{equivalent to computing the area:}$$

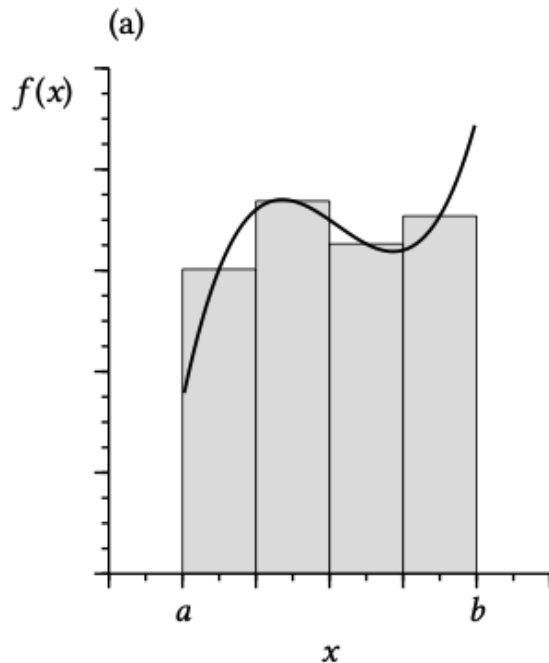


We need numerical integration when

- Cannot/difficult to integrate analytically
- Only know the integrand $f(x)$ at certain points

Rectangle rule

Easiest approach: approximate the area in the following way

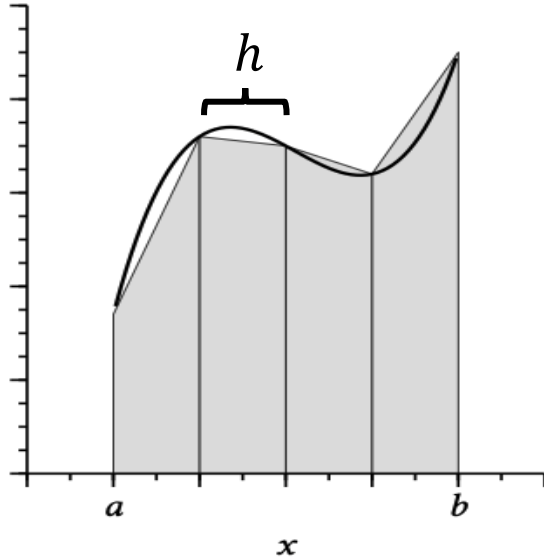


- 1) Sample the function at certain equidistant points
- 2) Consider rectangles with a width equal to the distance between two neighbor points and height equal to one value of function taken in that interval (can be e.g. the first value or the middle one)
- 3) Compute the rectangular areas and sum them up to obtain an approximated value of the area

The area however is not very close to the one of the curve

Trapezoidal rule

A better approach is to use trapezoids (**trapezoidal rule**)



1) Divide $[a, b]$ in N slices, each slice has a width $h = (b - a)/N$, so that each slice has a domain $[a + (k - 1)h, a + kh]$ with $k = 1, \dots, N$

2) The trapezoid in each slice is identified by the function's values at the border domain $f(a + (k - 1)h)$ and $f(a + kh)$ and has an area

$$A_k = \frac{1}{2}h[f(a + (k - 1)h) + f(a + kh)]$$

3) The total area is $I(a, b) \simeq \sum_{k=1}^N A_k = \frac{1}{2}h \sum_{k=1}^N [f(a + (k - 1)h) + f(a + kh)] = h[\frac{1}{2}f(a) + f(a + h) + f(a + 2h) + \dots + \frac{1}{2}f(b)]$

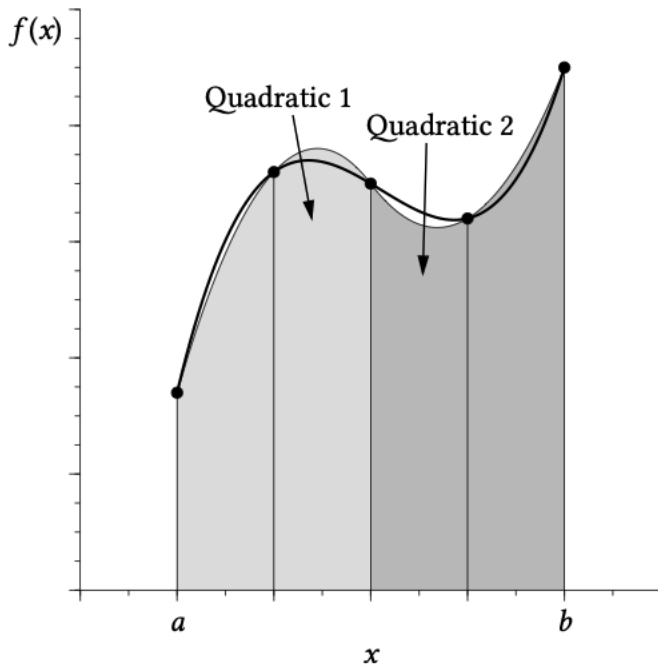
$$I(a, b) \simeq h \left[\frac{1}{2}f(a) + \frac{1}{2}f(b) + \sum_{k=1}^{N-1} f(a + kh) \right]$$

Larger N means smaller error
(to be discussed later)

Simpson's rule

The trapezoidal rule is often perfectly adequate for calculations where no great accuracy is required. But we can do a better job sometimes approximating the function with curves of some kind instead of straight lines.

For **Simpson's rule** we use quadratic curves:



- 1) take now three uniformly spaced points (or equivalently two adjacent slices)
- 2) fit a quadratic curve through the three points
- 3) compute the area under the quadratic curve
- 4) sum all estimated pair slice areas

Simpson's rule

Start fitting a quadratic curve under 3 points and compute the area

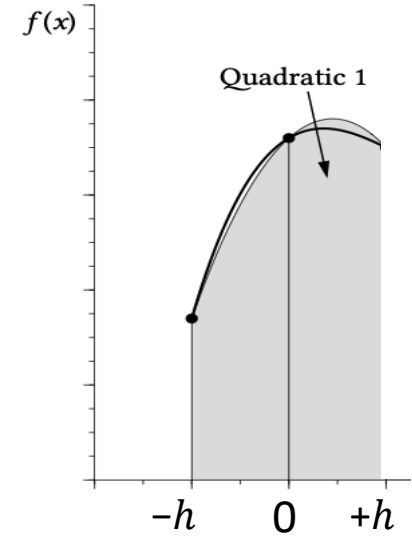
Consider $f(x)$, take three points at $x = -h, 0$, and $+h$ and fit a quadratic $Ax^2 + Bx + C$ so find A, B, C such that $f(x)$ at these three points correspond the quadratic function:

$$f(-h) = Ah^2 - Bh + C, \quad f(0) = C, \quad f(h) = Ah^2 + Bh + C.$$

Invert the equations and obtain A, B, C

$$A = \frac{1}{h^2} \left[\frac{1}{2}f(-h) - f(0) + \frac{1}{2}f(h) \right], \quad B = \frac{1}{2h} [f(h) - f(-h)], \quad C = f(0),$$

The area under the quadratic curve is $\int_{-h}^h (Ax^2 + Bx + C) dx = \frac{2}{3}Ah^3 + 2Ch = \frac{1}{3}h[f(-h) + 4f(0) + f(h)]$.



So we just need the values of $f(x)$ at the three evenly spaced points and input them in the formula. These can be any three points, not just $-h, 0, h$

Simpson's rule

Sum up the area of each successive double slice and obtain the total area (need even number of slices)

$$\begin{aligned} I(a, b) &\simeq \frac{1}{3}h[f(a) + 4f(a+h) + f(a+2h)] + \frac{1}{3}h[f(a+2h) + 4f(a+3h) + f(a+4h)] + \dots = \\ &= \frac{1}{3}h[f(a) + 4f(a+h) + 2f(a+2h) + 4f(a+3h) + \dots + f(b)] \end{aligned}$$

$$I(a, b) \simeq \frac{1}{3}h \left[f(a) + f(b) + 4 \sum_{\substack{k \text{ odd} \\ 1 \dots N-1}} f(a+kh) + 2 \sum_{\substack{k \text{ even} \\ 2 \dots N-2}} f(a+kh) \right]$$

For summation in Python, you can use directly “for k in range(1,N,2)” for the odd terms or “for k in range(2,N,2)” for even ones, or alternatively a single summation from 1 to N/2:

$$I(a, b) \simeq \frac{1}{3}h \left[f(a) + f(b) + 4 \sum_{k=1}^{N/2} f(a + (2k-1)h) + 2 \sum_{k=1}^{N/2-1} f(a + 2kh) \right]$$

Example

Integrate $x^4 - 2x + 1$ from $x=0$ to $x=2$ using $N=10, 100$ using the trapezoidal and Simpson's rules and compare to each other and the analytical value

```
#Trapezoidal rule
def f(x):
    return x**4 - 2*x + 1
N = 10
a = 0.0
b = 2.0
h = (b-a)/N
s = 0.5*f(a) + 0.5*f(b)
for k in range(1,N):
    s += f(a+k*h)
print(h*s)
```

```
#Simpson's rule
def f(x):
    return x**4 - 2*x + 1
N = 10          # need to be even
a = 0.0
b = 2.0
h = (b - a) / N
s = f(a) + f(b)
for k in range(1, N):
    xk = a + k * h
    if k % 2 == 0:
        s += 2 * f(xk)
    else:
        s += 4 * f(xk)
print((h / 3) * s)
```

$$\int_0^2 (x^4 - 2x + 1) dx = \left[\frac{1}{5}x^5 - x^2 + x \right]_0^2 = 4.4.$$

For $N=10$ we have $I=4.50656$ and $I=4.400426666$ for the trapezoidal and Simpson's method, to compare with 4.4, so an error of 2% and 0.0096%, while for $N=100$ $I = 4.40000004266$ and $I = 4.400000042666667$

Simpson's method reduces the error substantially, at the cost of only a few extra calculations.

Euler–Maclaurin error formula for trapezoidal rule

Aside from the rounding error, we have here a major error contribution, the **approximation error** as we are approximating an area in some form.

For the trapezoidal rule
Euler-Maclaurin rule

$$\int_a^b f(x) \, dx = \underbrace{\frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)]}_{\text{estimated integral value with this method}} + \underbrace{\frac{1}{12}h^2 [f'(a) - f'(b)]}_{\text{first Euler–Maclaurin correction term}} + O(h^4).$$

$\delta = \frac{1}{12}h^2 [f'(a) - f'(b)]$

δ is an estimation of the error in computing the integral. Note that the terms proportional to h^3 cancel out.

The trapezoidal rule is a first-order integration rule: it is accurate up to and including terms proportional to h and the leading-order approximation error is of order h^2 (**accurate to** $O(h)$ and has an **error** $O(h^2)$)

So one improves the result decreasing h . However, it can be decreased up to machine precision, where the rounding error starts to count

Euler–Maclaurin error formula for trapezoidal rule

Let's derive the error formula:

- divide interval $[a,b]$ in sample points $x_k = a + kh$
- Taylor expansion of $f(x)$ around x_{k-1} $f(x) = f(x_{k-1}) + (x - x_{k-1})f'(x_{k-1}) + \frac{1}{2}(x - x_{k-1})^2 f''(x_{k-1}) + \dots$
- Integrate from x_{k-1} to x_k

$$\int_{x_{k-1}}^{x_k} f(x) \, dx = f(x_{k-1}) \int_{x_{k-1}}^{x_k} dx + f'(x_{k-1}) \int_{x_{k-1}}^{x_k} (x - x_{k-1}) \, dx + \frac{1}{2} f''(x_{k-1}) \int_{x_{k-1}}^{x_k} (x - x_{k-1})^2 \, dx + \dots$$

- Substitute $u = x - x_{k-1}$ and perform integral

$$\begin{aligned} \int_{x_{k-1}}^{x_k} f(x) \, dx &= f(x_{k-1}) \int_0^h du + f'(x_{k-1}) \int_0^h u \, du + \frac{1}{2} f''(x_{k-1}) \int_0^h u^2 \, du + \dots \\ &= hf(x_{k-1}) + \frac{1}{2} h^2 f'(x_{k-1}) + \frac{1}{6} h^3 f''(x_{k-1}) + O(h^4), \end{aligned}$$

- Perform same operation expanding around x_k and integrating from x_{k-1} to x_k

$$\int_{x_{k-1}}^{x_k} f(x) \, dx = hf(x_k) - \frac{1}{2} h^2 f'(x_k) + \frac{1}{6} h^3 f''(x_k) - O(h^4).$$

Euler–Maclaurin error formula for trapezoidal rule

- Average the two results

$$\int_{x_{k-1}}^{x_k} f(x) \, dx = \frac{1}{2}h[f(x_{k-1}) + f(x_k)] + \frac{1}{4}h^2[f'(x_{k-1}) - f'(x_k)] + \frac{1}{12}h^3[f''(x_{k-1}) + f''(x_k)] + O(h^4).$$

- Sum over all slices

$$\int_a^b f(x) \, dx = \sum_{k=1}^N \int_{x_{k-1}}^{x_k} f(x) \, dx = \boxed{\frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)]} + \frac{1}{4}h^2[f'(a) - f'(b)] + \boxed{\frac{1}{12}h^3 \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)]} + O(h^4).$$

trapezoidal rule

trapezoidal rule applied to the function $f''(x)$

$$\int_a^b f''(x) \, dx = \frac{1}{2}h \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)] + O(h^2) \xrightarrow{\text{rearranging}} \frac{1}{12}h^3 \sum_{k=1}^N [f''(x_{k-1}) + f''(x_k)] = \frac{1}{6}h^2 \int_a^b f''(x) \, dx + O(h^4) = \frac{1}{6}h^2[f'(b) - f'(a)] + O(h^4),$$

This formula can be substituted back inside the integral

- We arrive at the Euler-MacLaurin formula

$$\int_a^b f(x) \, dx = \frac{1}{2}h \sum_{k=1}^N [f(x_{k-1}) + f(x_k)] + \frac{1}{12}h^2[f'(a) - f'(b)] + O(h^4).$$

The h^3 term cancels out, and one finds in general that only even powers of h survive

Error formula for Simpson's rule

For the Simpson's rule, the error would be

$$\delta = \frac{1}{180}h^4[f'''(a) - f'''(b)]$$

The error is $O(h^4)$, so two orders better than the trapezoidal rule.

As a rule of thumb, for Simpson's rule you don't need more than $N=10^4$ slides, limit in which you reach rounding error.

For the trapezoidal rule it would be $N=10^8$

Using one method over the other really depends on the integral: it is not always guaranteed that one method do better than the other, as we have a dependencies in δ over the derivatives values as well.

Practical estimation of error

What if we don't have access to $f(x)$ derivatives formula? Let's see a **practical method for trapezoid rule**.

Integrate using number of steps N_1 , and then double it $N_2 = 2N_1$, or equivalently halve the step:

$$h_1 = \frac{b-a}{N_1} \longrightarrow h_2 = \frac{h_1}{2}$$

Because the initial error scales as h_1^2 we effectively reduced the error to a quarter $h_2^2 = h_1^2/4$

$$\text{error} \propto h_1^2 \longrightarrow \propto \frac{h_1^2}{4}$$

And the real value I of an integral can be expressed as

$$I = I_1 + ch_1^2 \longrightarrow I = I_2 + ch_2^2$$

I_1, I_2 : integral estimates with the two steps
 c : common constant dependent only on f and not h (see MacLaurin formula)

Equating the two expressions:

$$I_1 + ch_1^2 = I_2 + ch_2^2, \longrightarrow I_2 - I_1 = ch_1^2 - ch_2^2 = 3ch_2^2, \longrightarrow \delta_2 = ch_2^2 = \frac{1}{3}(I_2 - I_1)$$

integral error estimate

Numerical integration: practical integrals error estimation

δ_2 is a simple way to estimate the error without using the Euler–Maclaurin formula

Rather than the signed value we take the absolute value

For trapezoidal rule

$$\delta_2 = \frac{1}{3} |I_2 - I_1|$$

For Simpson's rule

$$\delta_2 = \frac{1}{15} |I_2 - I_1|$$

Numerical integration: integrals error exercise

Take now the previous integral of $x^4 - 2x + 1$, estimate the error using the Euler–Maclaurin and the heuristic formulas we saw and compare with the error obtained directly using the correct integral

```
#Trapezoidal rule with error estimates
def f(x):
    return x**4 - 2*x + 1
def fp(x):
    return 4*x**3 - 2          # f'(x) for Euler–Maclaurin

def trapezoid(N, a, b):
    h = (b - a) / N
    s = 0.5*f(a) + 0.5*f(b)
    for k in range(1, N):
        s += f(a + k*h)
    return h*s, h

a, b = 0.0, 2.0
N = 10                                # fine grid (must be even if you also use N/2)

I2, h2 = trapezoid(N, a, b)          # I2 = T(h)
I1, h1 = trapezoid(N//2, a, b)      # I1 = T(2h)

# Heuristic leading error of I2:  $\delta_2 = c h^2 = (I2 - I1)/3$ 
delta2 = abs((I2 - I1) / 3.0)

# Euler–Maclaurin first correction for I2:  $(h^2/12)[f'(a) - f'(b)]$ 
em2 = abs((h2**2 / 12.0) * (fp(a) - fp(b)))

exactI = 4.4
print("T(N) = I2:", I2)
print("|I2-I|:", I2-exactI)
print("heuristic  $\delta_2 = (I2-I1)/3$ :", delta2)
print("EM correction on I2:", em2)

T(N) = I2: 4.50656
|I2-I|: 0.10655999999999999
heuristic  $\delta_2 = (I2-I1)/3$ : 0.10613333333333334
EM correction on I2: 0.10666666666666666
```

```
# Simpson's rule with error estimates
def f(x):
    return x**4 - 2*x + 1

def f3(x):
    return 24.0 * x            # f'''(x) for this f

def simpson(N, a, b):
    if N % 2 != 0:
        raise ValueError("N must be even")
    h = (b - a) / N
    s = f(a) + f(b)
    for k in range(1, N):
        xk = a + k*h
        s += 4*f(xk) if k % 2 else 2*f(xk)
    return (h/3.0)*s, h

a, b = 0.0, 2.0
N = 12                                # multiple of 4 so that N/2 is even

I2, h2 = simpson(N, a, b)            # I2 = S(h)
I1, h1 = simpson(N//2, a, b)        # I1 = S(2h)

# Heuristic leading error of I2:  $\delta_2 = c h^4 = (I2 - I1)/15$ 
delta2 = abs((I2 - I1) / 15.0)

# Euler–Maclaurin correction  $h^4/180 * [f'''(a) - f'''(b)]$ 
em2 = (h2**4) / 180.0 * (f3(a) - f3(b))

exactI = 4.4
print("S(N) = I2:", I2)
print("|I2-I|:", abs(I2 - exactI))
print("heuristic  $\delta_2 = (I2-I1)/15$ :", abs(delta2))
print("EM/Simpson correction on I2:", abs(em2))

S(N) = I2: 4.4002057613168715
|I2-I|: 0.0002057613168711143
heuristic  $\delta_2 = (I2-I1)/15$ : 0.0002057613168724141
EM/Simpson correction on I2: 0.00020576131687242795
```

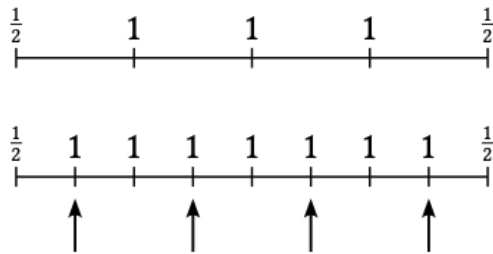
Choice of N : Adaptive integration

Suppose we want an accuracy of four decimal. In order e.g. to not waste computational resources, we want an N that gives this exact precision

Adaptive integration method:

- 1) Start estimating I with a small value of N
- 2) Double repeatedly N and estimate I and error.
- 3) Stop when error is smaller than target accuracy

For the trapezoidal rule, you also don't need to compute all $f(x_k)$ values for each N :



when doubling N , original points are still present. Need to add new ones with correct multipliers:

$$I_i = \frac{1}{2}I_{i-1} + h_i \sum_{\substack{k \text{ odd} \\ 1 \dots N_i-1}} f(a + kh_i)$$

In the book you will see the equivalent procedure for Simpson's rule

Higher order methods

Use higher-order polynomials to fit $f(x)$, e.g. with cubics, quartics, and so forth.

Consider the general form in which we wrote both trapezoidal and Simpson's rule:

$$\int_a^b f(x) \, dx \simeq \sum_{k=1}^N w_k f(x_k),$$

Degree	Polynomial	Coefficients
1 (trapezoidal rule)	Straight line	$\frac{1}{2}, 1, 1, \dots, 1, \frac{1}{2}$
2 (Simpson's rule)	Quadratic	$\frac{1}{3}, \frac{4}{3}, \frac{2}{3}, \frac{4}{3}, \dots, \frac{4}{3}, \frac{1}{3}$
3	Cubic	$\frac{3}{8}, \frac{9}{8}, \frac{9}{8}, \frac{3}{4}, \frac{9}{8}, \frac{9}{8}, \frac{3}{4}, \dots, \frac{9}{8}, \frac{3}{8}$
4	Quartic	$\frac{14}{45}, \frac{64}{45}, \frac{8}{15}, \frac{64}{45}, \frac{28}{45}, \frac{64}{45}, \frac{8}{15}, \frac{64}{45}, \dots, \frac{64}{45}, \frac{14}{45}$

what changes for each integration rule are the multiplicative coefficients: Newton–Cotes formulas

What we are actually doing is improving the fit of a portion of $f(x)$. Two possible tweaks we can implement:

- 1) with N sampled points, we could just fit $f(x)$ with a $(N-1)^{\text{th}}$ -degree polynomial to the whole integration interval, and get an integration method that is exact for $(N-1)^{\text{th}}$ -degree polynomials
- 2) uneven sampling space might give better results

These two tricks are used in the Gaussian quadrature method

Gaussian quadrature: fitting a polynomial

Let's now see **how to get the w_k coefficients** of this formula in the general case of **uneven sampling** of the x_k points

$$\int_a^b f(x) dx \simeq \sum_{k=1}^N w_k f(x_k),$$

- 1) Fit a single polynomial of degree $N-1$ through the values $f(x_k)$
- 2) Integrate the polynomial in $[a, b]$

Let's first define the **Lagrange basis** $\phi_k(x)$ (degree $N-1$ in x)

$$\phi_k(x) = \prod_{\substack{m=1 \dots N \\ m \neq k}} \frac{(x - x_m)}{(x_k - x_m)} = \frac{(x - x_1)}{(x_k - x_1)} \times \dots \times \frac{(x - x_{k-1})}{(x_k - x_{k-1})} \times \frac{(x - x_{k+1})}{(x_k - x_{k+1})} \times \dots \times \frac{(x - x_N)}{(x_k - x_N)}$$

Property: it is zero for all x_m except x_k , for which is 1: $\phi_k(x_m) = \delta_{km}$.

We now define this $N-1$ degree polynomial as a combination of the $\phi_k(x)$:

$$\Phi(x) = \sum_{k=1}^N f(x_k) \phi_k(x)$$

Lagrange interpolating polynomial

Gaussian quadrature: fitting a polynomial

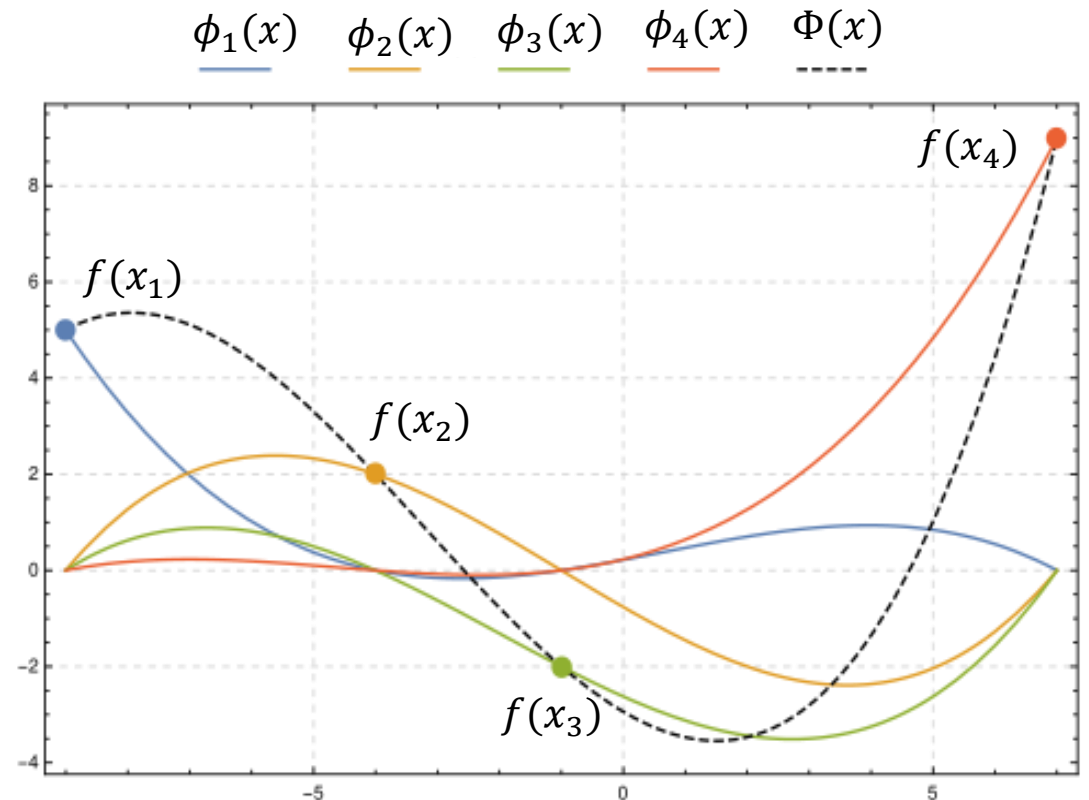
The Lagrange polynomial has the property that at each sampled point x_m it gives the exact value of $f(x_m)$:

$$\Phi(x_m) = \sum_{k=1}^N f(x_k) \phi_k(x_m) = \sum_{k=1}^N f(x_k) \delta_{km} = f(x_m),$$

An example (from Wikipedia):

Each Lagrange basis $\phi_k(x)$ passes only through $f(x_k)$

The Lagrange polynomial (dashed line) passes through all points



Gaussian quadrature: fitting a polynomial

$\Phi(x)$ is exactly the polynomial we wanted. A $N-1$ degree polynomial of that fits the integrand $f(x)$ at all of the sample points. It is also unique.

$$\Phi(x) = \sum_{k=1}^N f(x_k) \phi_k(x)$$

Integrating over $[a,b]$:

$$\int_a^b f(x) \, dx \simeq \int_a^b \Phi(x) \, dx = \int_a^b \sum_{k=1}^N f(x_k) \phi_k(x) \, dx = \sum_{k=1}^N f(x_k) \int_a^b \phi_k(x) \, dx,$$

So for this method

$$\int_a^b f(x) \, dx \simeq \sum_{k=1}^N w_k f(x_k),$$

with

$$w_k = \int_a^b \phi_k(x) \, dx.$$

Gaussian quadrature: fitting a polynomial

It remains to specify $f(x)$ and compute the w_k either analytically, or using the numerical integration methods.

The advantage is that I can just compute the w_k once for a specific interval, say $[-1,1]$

$$w_k = \int_{-1}^1 \phi_k(x) dx.$$

Then if I want to compute the integral in $[a,b]$ I just perform a change in variable

$$x'_k = \frac{1}{2}(b-a)x_k + \frac{1}{2}(b+a).$$

giving $\int_a^b f(x) dx \simeq \sum_{k=1}^n w'_k f(x'_k).$ with $w'_k = \frac{1}{2}(b-a)w_k.$

Gaussian quadrature: choice of sampling points x_k

Now we want to create a rule that gives the exact integral of any polynomial function $f(x)$ up to degree $2N-1$ with just N sample points.

We will get an automatic choice for the position of the sampling points x_k and of the coefficient w_k .

$$\int_{-1}^1 f(x) dx = \sum_{k=1}^N w_k f(x_k), \quad (\text{we restrict to } [-1,1], \text{ but we saw we can extend to any interval})$$

First we need to define the **Legendre polynomial** $P_N(x)$.

$P_N(x)$ is a N^{th} -degree polynomial such that:

1) It satisfies the following orthogonality condition

$$\int_{-1}^1 x^k P_N(x) dx = 0 \quad \text{for all integer } k \text{ in the range } 0 \leq k < N$$

2) $P_N(1) = 1$.

Gaussian quadrature: choice of sampling points x_k

Using these two rules, one can obtain recursively the Legendre polynomials

Start from $P_0(x)$:

we need $P_0(x) = \text{const}$, while using $P_0(1) = 1$ means $P_0(x) = 1$

For $P_1(x)$, we need $P_1(x) = ax + b$, and has to be orthogonal to $P_0(x)$ and $P_1(1) = 1$:

$$\int_{-1}^1 (ax + b) dx = 0 \quad \longrightarrow \quad b = 1 \quad \text{and} \quad P_1(1) = a = 1 \quad \text{thus} \quad P_1(x) = x$$

And so on

$$P_0(x) = 1;$$

$$P_1(x) = x;$$

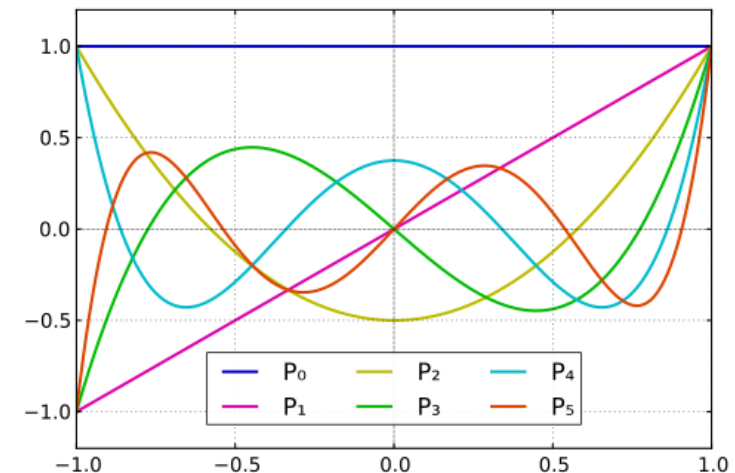
$$P_2(x) = \frac{1}{2}(3x^2 - 1);$$

$$P_3(x) = \frac{1}{2}(5x^3 - 3x);$$

$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3);$$

$$P_5(x) = \frac{1}{8}(63x^5 - 70x^3 + 15x);$$

$$P_6(x) = \frac{1}{16}(231x^6 - 315x^4 + 105x^2 - 5).$$



Gaussian quadrature: choice of sampling points x_k

An important property of $P_N(x)$ is that it is **orthogonal to any polynomial $q(x)$ of degree $N-1$** (using the orthogonality condition):

given $q(x) = \sum_{k=0}^{N-1} c_k x^k$ with c_k some set of coefficients

$$\int_{-1}^1 q(x) P_N(x) dx = \sum_{k=0}^{N-1} c_k \int_{-1}^1 x^k P_N(x) dx = 0,$$

Another property is that $P_N(x)$ **has N real roots that all lie in the interval from -1 to 1**

Now we use the Legendre polynomials to find a formula for the integral of a polynomial function $f(x)$:

$$\int_{-1}^1 f(x) dx = \sum_{k=1}^N w_k f(x_k),$$

Gaussian quadrature: choice of sampling points x_k

Suppose the **integrand** $f(x)$ is a **polynomial in x of degree $2N-1$** or less, then it can be expressed as

$$f(x) = q(x)P_N(x) + r(x), \quad \text{with } q(x) \text{ and } r(x) \text{ both polynomials of degree } N-1 \text{ or less}$$

Thus
$$\int_{-1}^1 f(x) dx = \int_{-1}^1 q(x)P_N(x) dx + \int_{-1}^1 r(x) dx = \int_{-1}^1 r(x) dx,$$

 $q(x)$ is orthogonal to $P_N(x)$ being of less degree

So to find $\int_{-1}^1 f(x) dx$ we have only to find $\int_{-1}^1 r(x) dx$, which has degree $N-1$ or less

Gaussian quadrature: choice of sampling points x_k

Now remember that for any choice of N sample points x_k of the $r(x)$ polynomial (with $N-1$ degree or less) a Lagrange polynomial $\Phi(x)$ can be fitted exactly. Because the polynomial is unique, $\Phi(x) = r(x)$

$$\int_{-1}^1 f(x) \, dx = \int_{-1}^1 r(x) \, dx = \int_{-1}^1 \Phi(x) \, dx = \underbrace{\sum_{k=1}^N r(x_k) \phi_k(x)}_{\text{definition of Lagrange polynomial}} = \sum_{k=1}^N w_k r(x_k), \quad \text{with} \quad w_k = \int_{-1}^1 \phi_k(x) \, dx.$$

definition of Lagrange polynomial

We have now an exact expression of the $2N-1$ degree or less polynomial $f(x)$ in terms of the interpolating polynomials $\phi_k(x)$. But we still need to compute $r(x)$.

However, we can have the freedom to choose the x_k points.

Gaussian quadrature: choice of sampling points x_k

Say we **choose the zeros of $P_N(x)$** , which are exactly N and between -1 and 1

$$f(x) = q(x)P_N(x) + r(x),$$
$$\sum_{k=1}^N w_k f(x_k) = \sum_{k=1}^N w_k q(x_k) \underset{P_N(x_k) = 0}{P_N(x_k)} + \sum_{k=1}^N w_k r(x_k) = \sum_{k=1}^N w_k r(x_k) = \int_{-1}^1 f(x) dx$$

We have the exact integration rule for any polynomial function $f(x)$ of degree $2N-1$, if we choose as x_k the zeros of $P_N(x)$

$$\int_{-1}^1 f(x) dx = \sum_{k=1}^N w_k f(x_k),$$

and we will now see we don't actually need to compute $r(x)$, just the position of the zeros

Gaussian quadrature: weights w_k

Let's see a formula to express the weights $w_k = \int_a^b \phi_k(x) dx$.

Recall the Lagrange interpolating polynomial, where the x_m are the zeros of $P_N(x)$ $\phi_k(x) = \prod_{\substack{m=1 \dots N \\ m \neq k}} \frac{(x - x_m)}{(x_k - x_m)}$.

By definition $P_N(x)$ can be expressed in terms of its zeros

$$P_N(x) = A \prod_{m=1}^N (x - x_m) = (x - x_k) \psi_k(x) \quad \text{with} \quad \psi_k(x) = A \prod_{\substack{m=1 \dots N \\ m \neq k}} (x - x_m),$$

We can express $\phi_k(x) = \frac{\psi_k(x)}{\psi_k(x_k)} = \frac{P_N(x)/(x - x_k)}{P'_N(x_k)}$,

using l'Hopital's rule $\psi_k(x_k) = \lim_{x \rightarrow x_k} \frac{P_N(x)}{x - x_k} = P'_N(x_k)$

Gaussian quadrature: weights w_k

The weights become

$$w_k = \int_{-1}^1 \phi_k(x) dx = \frac{1}{P'_N(x_k)} \int_{-1}^1 \frac{P_N(x)}{x - x_k} dx = \frac{2}{NP'_N(x_k)P_{N-1}(x_k)}.$$

see book for demonstrations

$$\int_{-1}^1 \frac{P_N(x)}{x - x_k} dx = \frac{2}{NP_{N-1}(x_k)}. \quad P'_N(x_k) = \frac{NP_{N-1}(x_k)}{1 - x_k^2}$$

We arrive at the formulas for w_k

$$w_k = \frac{2}{(1 - x_k^2)[P'_N(x_k)]^2},$$

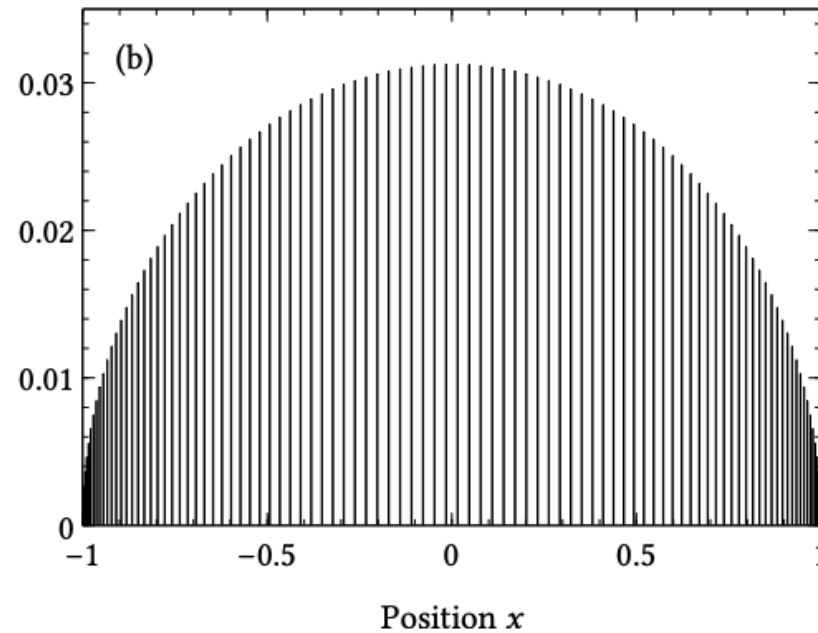
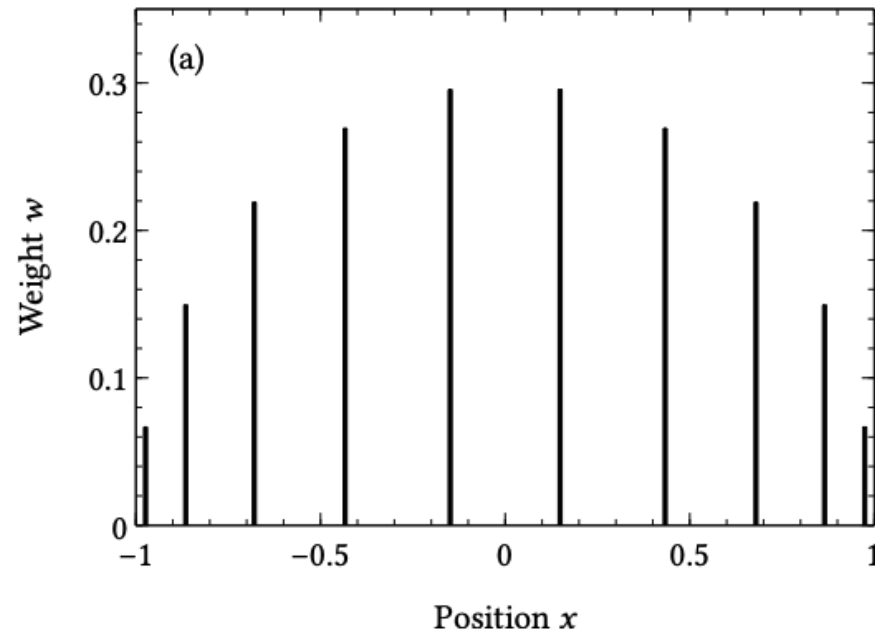
$$w_k = \frac{2(1 - x_k^2)}{[NP_{N-1}(x_k)]^2}.$$

the second is more practical if one derives iteratively $P_N(x)$ from the recurrence relation

$$(n + 1)P_{n+1}(x) = (2n + 1)xP_n(x) - nP_{n-1}(x),$$

Gaussian quadrature

Sample points and weights for $N=10$ and $N=100$.



Note how the points get closer together at the edges while at the same time the weights get smaller

Gaussian quadrature

The derivation is complex, but the **implementation** is sample.

Start from any function $f(x)$ in $[-1,1]$ (not necessarily a polynomial)

1) choose the number of sample points N

2) compute $P_N(x)$, its zeros x_k and the weights $w_k = \frac{2(1 - x_k^2)}{[NP_{N-1}(x_k)]^2}$

3) compute the integral as $\int_{-1}^1 f(x) dx \simeq \sum_{k=1}^N w_k f(x_k)$

For integrals over any other interval, rescale x_k and w_k :

$$x'_k = \frac{1}{2}(b - a)x_k + \frac{1}{2}(b + a).$$

$$w'_k = \frac{1}{2}(b - a)w_k.$$

It can be demonstrated that the integral converges very fast, improves by a factor of c/N^2 when increasing the number of samples by just one (c depending on the function)

Disadvantages:

- x_k change increasing N : we cannot reuse the same $f(x_k)$ doubling N , in order to estimate the error
- $f(x)$ should not vary a lot in between sampling points

Code for $P_N(x)$ zeros and w_k

This is how a working code to compute the $P_N(x)$ and its zeros x_k and the w_k

```
def gaussxw(N,a=-1.0,b=1.0):  
  
    # If N<=200 use look-up tables, else use Newton's method  
    if N<=200:  
  
        x,w = array(xlist[N],float),array(wlist[N],float)  
  
    else:  
  
        # Initial approximation to roots of the Legendre polynomial  
        a = longdouble(linspace(3,4*N-1,N)/(4*N+2))  
        x = cos(pi*a+1/(8*N*N*tan(a)))  
  
        # Find roots using Newton's method  
        epsilon = 1e-18  
        delta = 1.0  
        while delta>epsilon:  
            p0 = ones(N,longdouble)  
            p1 = copy(x)  
            for k in range(1,N):  
                p0,p1 = p1,((2*k+1)*x*p1-k*p0)/(k+1)  
            dp = (N+1)*(p0-x*p1)/(1-x*x)  
            dx = p1/dp  
            x -= dx  
            delta = max(abs(dx))  
  
        # Calculate the weights (updated to new formula 4/15/25)  
        # w = 2*(N+1)*(N+1)/(N*N*(1-x*x)*dp*dp)  
        w = 2*(1-x*x)/(N*N*p0*p0)  
  
    return 0.5*(b-a)*x+0.5*(b+a),0.5*(b-a)*w
```

tabulated data for small N

Initialize array of x_k with initial guess for zeros

$$x_k \approx \cos\left(\pi a_k + \frac{1}{8N^2 \tan a_k}\right), \quad a_k = \frac{3, 7, \dots, 4N-1}{4N+2}.$$

Compute the value of $P_N(x_k)$ iteratively, starting from $N=0$ (for which $P_0(x_k)=1$ and $P_1(x_k)=x$) using

$$P_{k+1}(x) = \frac{(2k+1)xP_k(x) - kP_{k-1}(x)}{k+1}$$

Newton's method to change position of x_k . Compute derivative and check variation of position dx (becomes zero when $P_N(x_k)=0$). The variation tells where you move next the guess

$$P'_N(x) = \frac{(N+1)}{1-x^2} (P_{N-1}(x) - xP_N(x))$$

returns the two arrays x_k and w_k in the given range $[a,b]$

Example of Gaussian quadrature

This is how to use the code as a function

```
#Gaussian quadrature method
from gaussxw import gaussxw
def f(x):
    return x**4 - 2*x + 1
N = 3
a = 0.0
b = 2.0
# Calculate the sample points and weights
x,w = gaussxw(N,a,b)
# Perform the integration
s = 0.0
for k in range(N):
    s += w[k]*f(x[k])
print(s)
```

4.4

We have the exact result with just 3 sample points; even though it's a 4th-degree polynomial (we proved we can compute the exact integral of any polynomial of $2N-1$ degree with just N points). So even if 3 points are not enough to fit a polynomial, they are enough to compute its exact integral.

Numerical integration: which method to use?

- Trapezoidal rule
 - Good for quick calculations not requiring great accuracy
 - Does not rely on the integrand being smooth; a good choice for **noisy/singular integrands, equally spaced points**
- Simpson's rule
 - Similar benefits to trapezoidal rule, but better accuracy and faster.
 - Can lead to problems if the integrand is noisy or otherwise not smooth
- Gaussian quadrature
 - **Theoretically most accurate if the function is relatively smooth**
 - **Good for many repeated calculations of the same type of integral**
 - Requires unequally spaced nodes
 - **Error can be challenging to control, especially for non-smooth functions**
 - **Bad for discontinuous integrands**

Infinite ranges

What about integrals with infinite ranges? $\int_0^{\infty} f(x) \, dx$.

Do a variable change: $z = \frac{x}{1+x}$

The integral then take values between $[0,1]$ and we can apply any method

$$\int_0^{\infty} f(x) \, dx = \int_0^1 \frac{1}{(1-z)^2} f\left(\frac{z}{1-z}\right) dz,$$

$$x = \frac{z}{1-z}.$$

$$dx = dz/(1-z)^2$$

Other valid transformations if the first one does not a good job:

$$z = \frac{x}{c+x}$$

$$z = x^y/(1+x^y)$$

Infinite ranges

If the integral is in the range $[a, +\infty]$ you can add first a shift to the transformation

$$\left. \begin{array}{l} y = x - a, \\ z = y/(1 + y) \end{array} \right\} \quad z = \frac{x - a}{1 + x - a} \quad \longrightarrow \quad \int_a^\infty f(x) \, dx = \int_0^1 \frac{1}{(1 - z)^2} f\left(\frac{z}{1 - z} + a\right) \, dz.$$

If it goes from $[-\infty, a]$

$$z = \frac{1}{1 - x + a} \quad \longrightarrow \quad \int_{-\infty}^a f(x) \, dx = \int_0^1 \frac{1}{z^2} f\left(\frac{z - 1}{z} + a\right) \, dz.$$

If it goes from $[-\infty, +\infty]$ we can split the integral into two parts, or change the variable as

$$x = \frac{z}{1 - z^2}, \quad \longrightarrow \quad \int_{-\infty}^\infty f(x) \, dx = \int_{-1}^1 \frac{1 + z^2}{(1 - z^2)^2} f\left(\frac{z}{1 - z^2}\right) \, dz.$$

$$x = \tan z, \quad \longrightarrow \quad \int_{-\infty}^\infty f(x) \, dx = \int_{-\pi/2}^{\pi/2} \frac{f(\tan z)}{\cos^2 z} \, dz.$$

Numerical integration : example with infinite range

Calculate the value of the following integral using Gaussian quadrature $I = \int_0^{\infty} e^{-t^2} dt.$

Using the change in variable $z = \frac{x}{1+x} \longrightarrow I = \int_0^1 \frac{e^{-z^2/(1-z)^2}}{(1-z)^2} dz.$

```
#example with infinite range
from gaussxw import gaussxw
from math import exp

def f(z):
    return exp(-z**2/(1-z)**2)/(1-z)**2

N = 50
a = 0.0
b = 1.0
x,w = gaussxw(N,a,b)
s = 0.0
for k in range(N):
    s += w[k]*f(x[k])
print(s)
```

0.8862269254528359

In fact $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} = 1.772454 \dots$ and $\frac{1}{2}\sqrt{\pi} = 0.8862269254527580\dots$

Multiple integrals

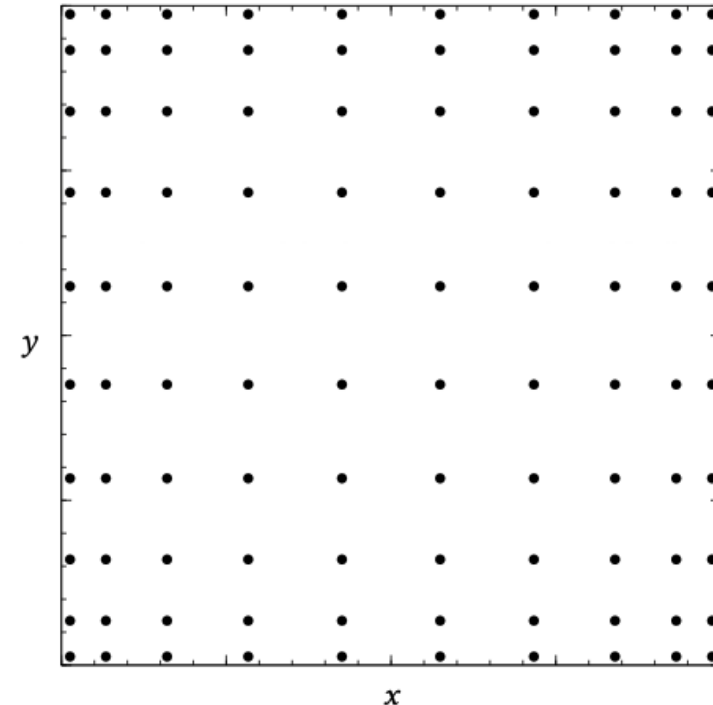
Consider multi-variable integrals $I = \int_0^1 \int_0^1 f(x, y) \, dx \, dy.$

Defining $F(y) = \int_0^1 f(x, y) \, dx.$ we can rewrite it as $I = \int_0^1 F(y) \, dy.$

So, one way would be to first to evaluate $F(y)$ for a suitable set of y , then compute I

We could use Gaussian quadrature for example

$$F(y) \simeq \sum_{i=1}^N w_i f(x_i, y) \quad \text{and} \quad I \simeq \sum_{j=1}^N w_j F(y_j).$$



Multiple integrals

Substituting the first sum into the second we get

$$I \simeq \sum_{i=1}^N \sum_{j=1}^N w_i w_j f(x_i, y_j).$$

a kind of 2D version of Gaussian quadrature, with weights $w_i w_j$

However, in 2D there is no real reason to choose sample points on a grid, and no general argument why Gaussian quadrature would be the best choice

What about random points? -> Monte-Carlo methods

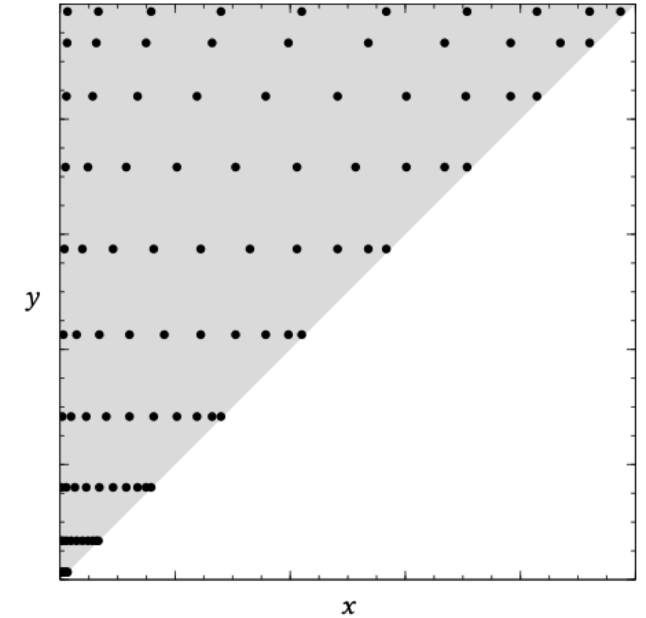
Numerical integration : multiple integrals

Another type of multiple integral: $I = \int_0^1 dy \int_0^y dx f(x, y).$

Can use same approach as before

$$F(y) = \int_0^y f(x, y) dx, \quad \text{and} \quad I = \int_0^1 F(y) dy,$$

If we use now Gaussian quadrature, we will get uneven sampling, which is not ideal because some parts will be less accurate than others.



What about strange integration domain shapes?

Use a normal grid and set function to zero outside domain
Or use random points

